SOFTWARE ARCHITECTURE

Software architecture refers to the high-level structure of a software system, defining how its components interact to fulfil system requirements. It represents the blueprint for both system organization and communication between elements such as modules, components, and data flows. Formally, software architecture is "the set of structures needed to reason about the system, comprising software elements, their relations, and the properties of both."

Purpose and Importance

Provides a conceptual framework for system design and evolution.
Ensures scalability, maintainability, and performance optimization.
Enables stakeholder communication between developers, clients, and managers.
Guides technology selection and development strategy.

Architectural Views

Logical View: Focuses on functionality for end users.
Process View: Describes the system's runtime behavior and concurrency.
Development View: Shows software modules and their organization in the development environment.
Physical View: Describes deployment on hardware.

Architectural Styles and Patterns

A well-chosen architectural pattern balances functionality, performance, and system flexibility. Common architectural styles include:

Layered Architecture: System divided into presentation, business logic, and data layers.
Client-Server Architecture: Tasks divided between clients (requesters) and servers (providers).
Service-Oriented Architecture (SOA): Functions offered as independent services accessible via network.
Event-Driven Architecture: Components react to events or messages asynchronously.
Microservices Architecture: Independent services work together through lightweight APIs.

SOFTWARE QUALITY AND TESTING

Software quality refers to the degree to which a software product meets its specified requirements and user expectations. It is measured by attributes such as reliability, usability, efficiency, and maintainability. According to ISO/IEC 25010, quality is evaluated across key dimensions like functionality, performance, compatibility, security, and portability.

Importance of Quality

Reduces maintenance cost.
Enhances user satisfaction.
Builds trust and reputation for the organization.
Minimizes failure risk and downtime.

Software Testing
Testing is the systematic process of evaluating software to detect defects and verify that it behaves as expected. A strong testing framework is essential for achieving and maintaining high software quality.

Goals of Testing:
Identify defects before deployment.
Ensure compliance with functional and non-functional requirements.
Validate reliability and performance.

Levels of Testing
Unit Testing: Testing individual components or modules.
Integration Testing: Checking interactions between integrated components.
System Testing: Validating the complete system as a whole.
Acceptance Testing: Ensuring the software meets user requirements before release.

Types of Testing
Black-Box Testing: Based on input-output behavior; internal logic ignored.
White-Box Testing: Based on internal structure and code paths.
Regression Testing: Ensures new updates don't break existing features.
Performance Testing: Evaluates speed, scalability, and resource use.
Security Testing: Ensures data integrity and protection.

SOFTWARE VALIDATION
Software validation is the process of confirming that the developed software meets user needs and intended use, not just technical specifications. It asks the question: "Are we building the right product?" In contrast, verification asks: "Are we building the product right?" Together, these ensure both correctness and relevance.

Validation Process
Requirements Validation: Confirm requirements are clear, consistent, and achievable.
Design Validation: Ensure design satisfies all functional requirements.
Code Validation: Use testing and inspection to confirm correctness.
User Acceptance Validation: Obtain user feedback to confirm satisfaction.

Validation Techniques
Reviews and Walkthroughs: Early manual checking of design and code.
Prototyping: Building simplified versions to confirm requirements.
Simulations: Testing behaviour in controlled environments.
User Acceptance Testing (UAT): Real users validate the final product.

Importance

Reduces rework by catching issues early.
Builds user confidence in system reliability.
Ensures regulatory or organizational compliance.
Supports long-term maintainability and evolution.
When integrated, they create a feedback-driven development process, ensuring software is not only functional but also robust, measurable, and aligned with real-world goals.